

如何微调BERT模型进行文本分类

20分钟内，从零到一构建高精度中文情感分析模型



你是否也遇到过这些烦恼？

上万条用户评论，还在手动分析？
眼睛都快看瞎了...

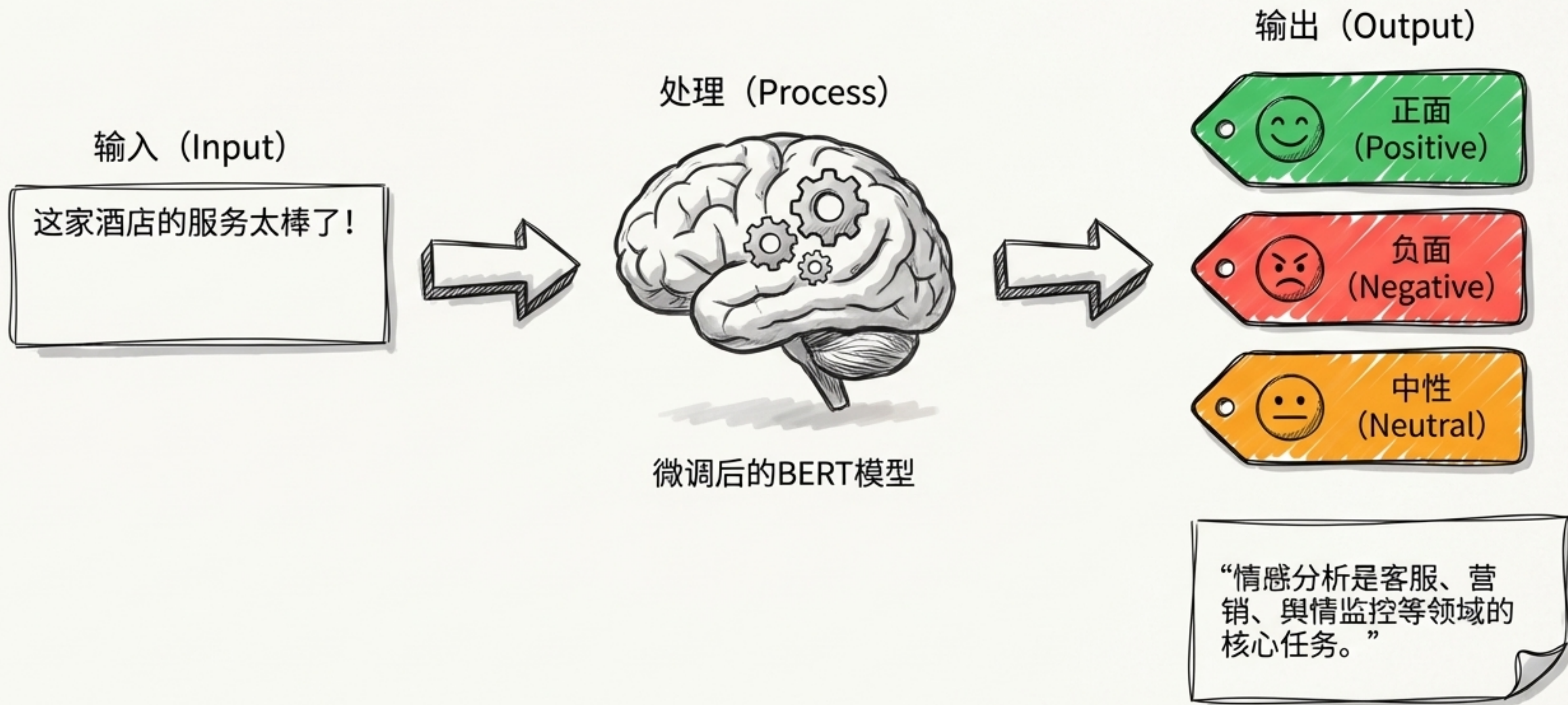


传统模型准确率上不去？
负面评价被当成正面，差点酿成大祸！



今天，这套方案就是你的“救星”。✓

我们的目标：构建一个智能情感分析系统

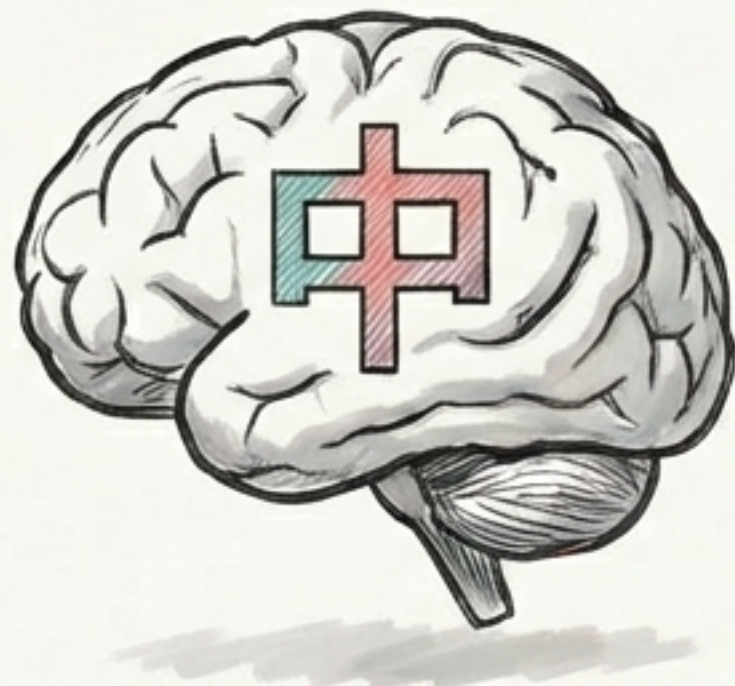


为什么选择BERT微调？



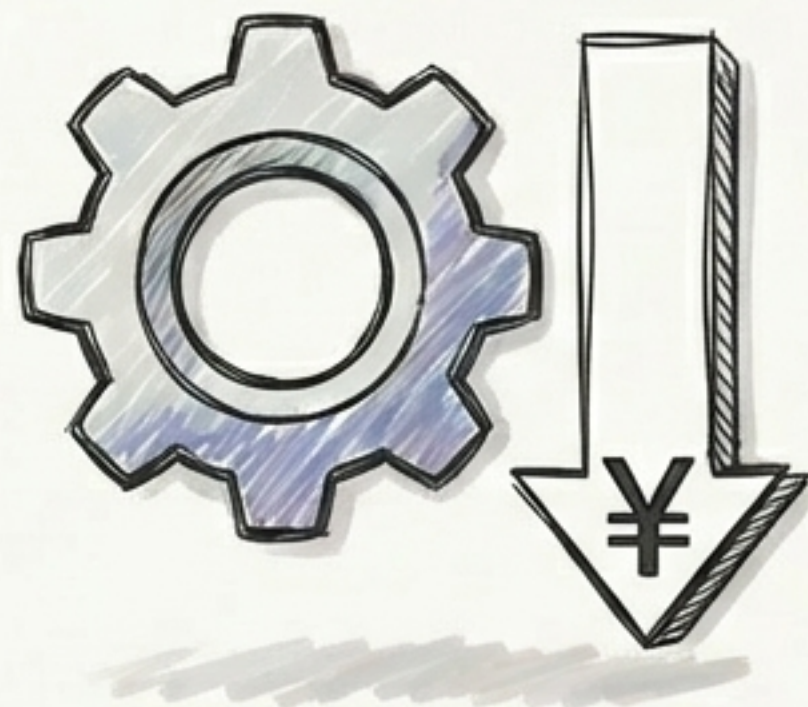
双向理解

精确捕捉上下文语境，理解更深层含义。



中文预训练

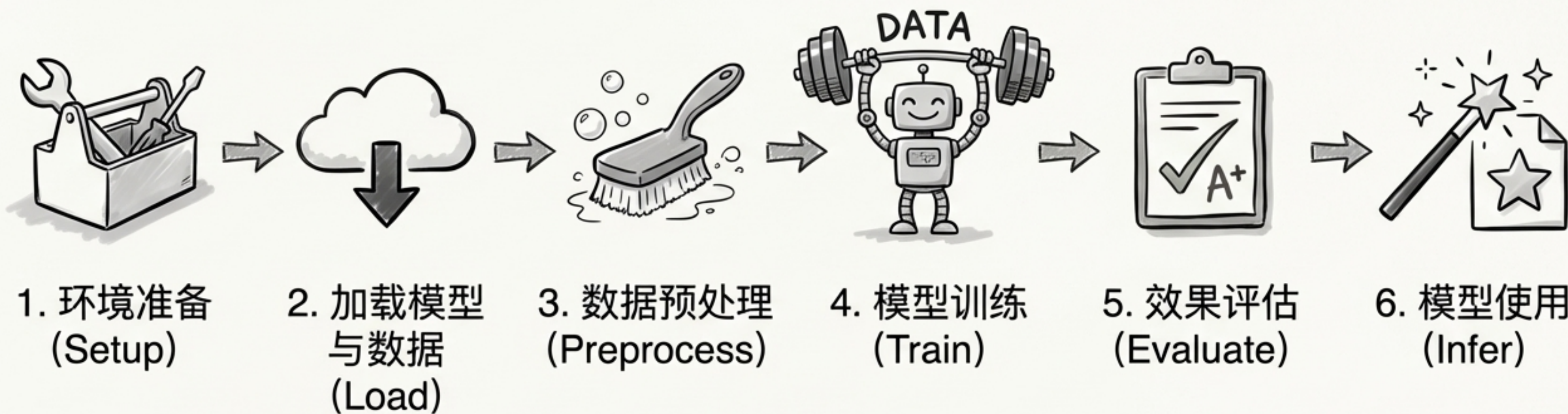
模型已经“学会”了中文，我们只需进行任务适配。



高效低成本

无需从零开始训练，极大降低了算力和时间成本。

微调BERT的全流程蓝图



这个流程不仅适用于情感分析，更是所有微调任务的通用模板。

本次分享大纲



第一部分：项目解析

- 明确“做什么”与“为什么做”



第二部分：实战演练：代码逐行讲解

- Step 1: 环境与依赖安装
- Step 2: 加载模型与分词器
- Step 3: 加载与处理数据集
- Step 4: 配置与启动训练



第三部分：效果验证

- 微调前后的惊人对比



第四部分：总结与提升

- 如何让你的模型更上一层楼

实战演练（一）：准备我们的“工具箱”

安装项目所需的核心库

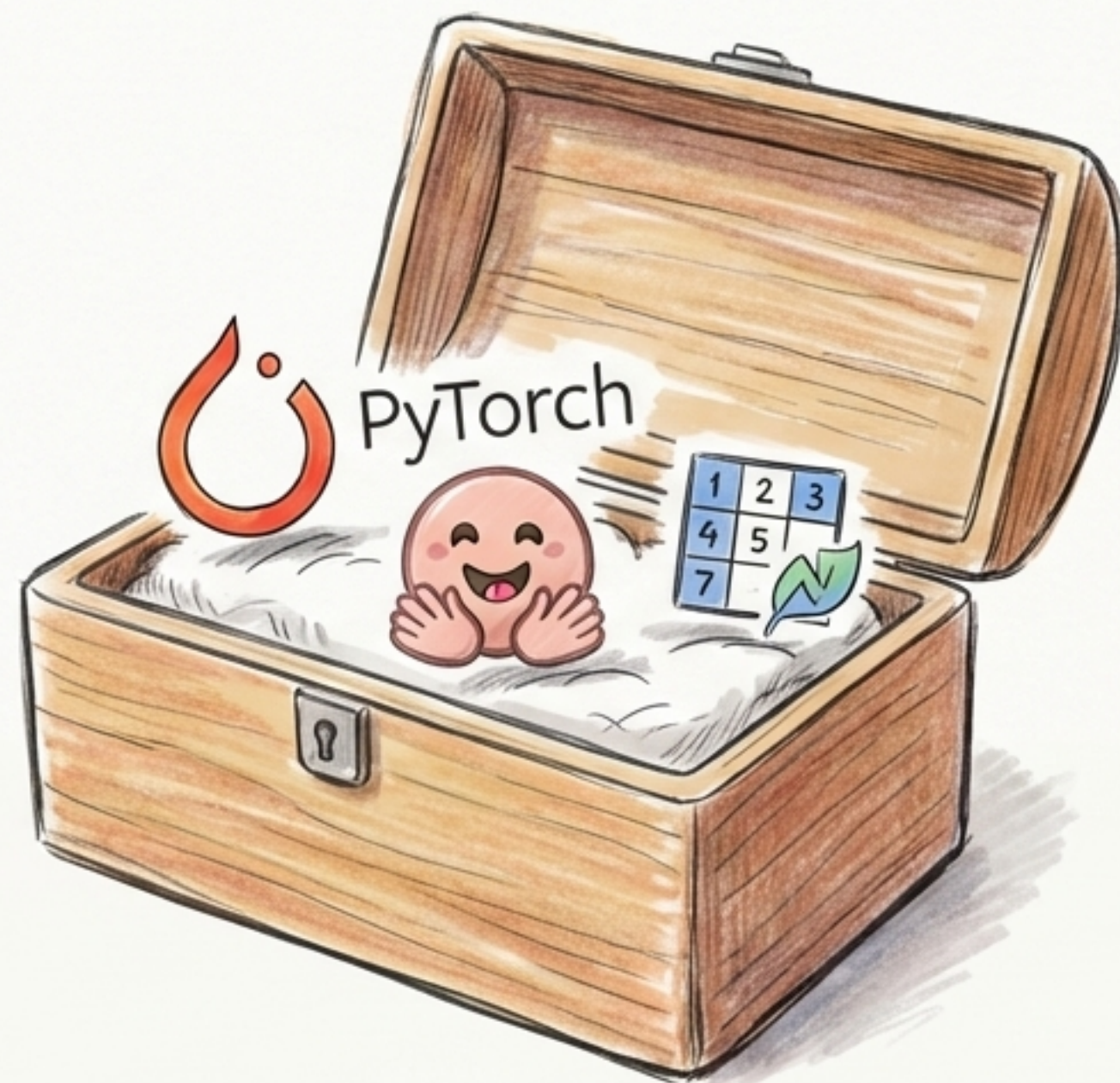
```
pip install torch —————> # 深度学习框架核心
```

```
pip install transformers # Hugging Face库, 用于加载BERT模型
```

```
pip install datasets —————> # 轻松加载和处理数据集
```

```
pip install evaluate —————> # 用于计算准确率等评估指标
```

```
pip install pandas numpy # 数据处理和科学计算的基础
```

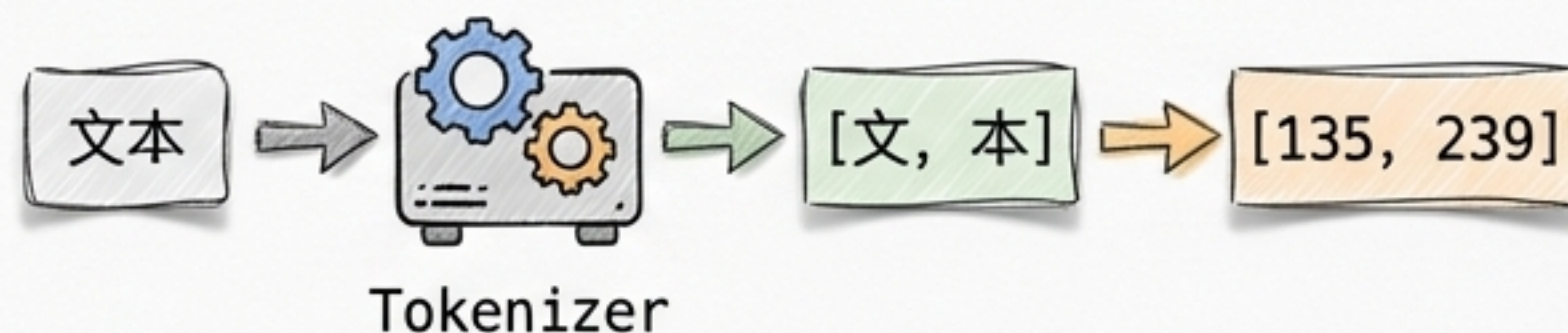


实战演练（二）：加载模型与“翻译官”（分词器）

```
from transformers import AutoTokenizer,
AutoModelForSequenceClassification

model_name = "bert-base-chinese"
tokenizer = AutoTokenizer.from_pretrained(
    model_name)
model = AutoModelForSequenceClassification.
from_pretrained(
    model_name, num_labels=3
)
```

什么是分词器（Tokenizer）？



它就像一个翻译官，把计算机无法理解的文字，转换成可以运算的数字ID。

num_labels=3 的含义？

告诉模型我们的任务是三分类：正面、负面、中性。

实战演练（三）：准备“养料” - 数据集

Data Structure

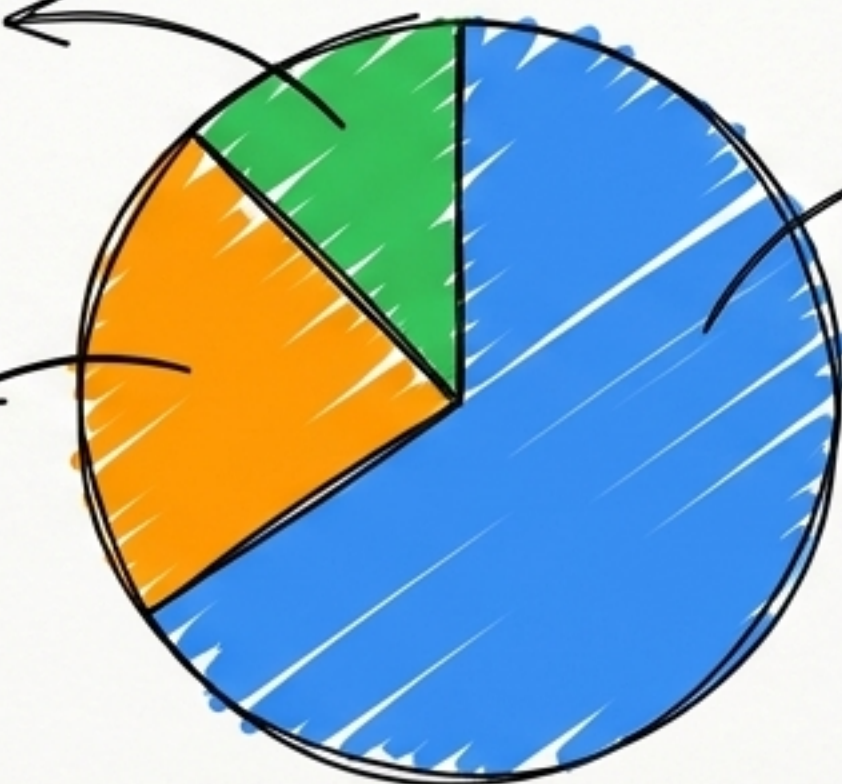
```
from datasets import load_dataset
raw_datasets = load_dataset("seamew/ChnSentiCorp")
```

text	label
“这家酒店的风景真美...”	1（积极）
“房间隔音太差了，一晚上没睡好。”	0（消极）

数据划分的重要性

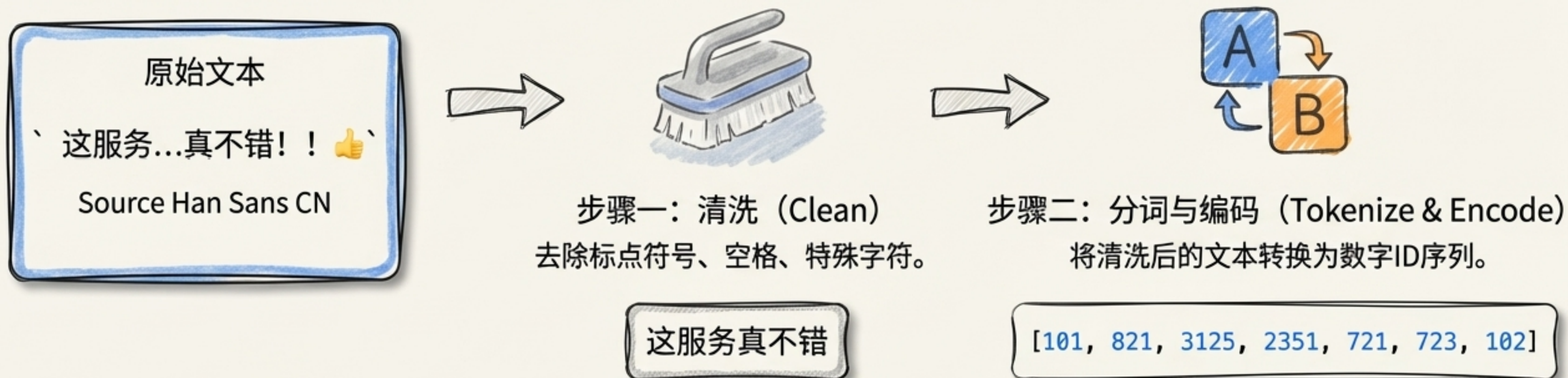
测试集 (Test Set) :
用于最终评估模型性能。

验证集 (Validation Set) :
用于训练中调整参数。



训练集 (Train Set) :
用于模型学习。（9600条）

实战演练（四）：数据清洗与编码



```
def preprocess_function(examples):  
    # 1. 清洗文本 (伪代码)  
    cleaned_text = clean(examples["text"])  
    # 2. 调用tokenizer进行编码  
    return tokenizer(cleaned_text, truncation=True)  
  
tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

实战演练（五）：设置训练参数 - 指挥模型如何学习

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,           # 训练轮次
    per_device_train_batch_size=16, # 每批次大小
    evaluation_strategy="epoch",  # 每轮评估一次
    logging_dir='./logs',
)
```



让模型把全部数据看几遍？这里设置为3遍。



每次喂给模型多少条数据？设置为16条。



多久检查一次学习效果？设置为每一轮结束时。

实战演练（六）：集结部队，开始训练！

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    tokenizer=tokenizer,
    compute_metrics=compute_accuracy, # 定义好的
    评估函数
)
```

trainer.train()

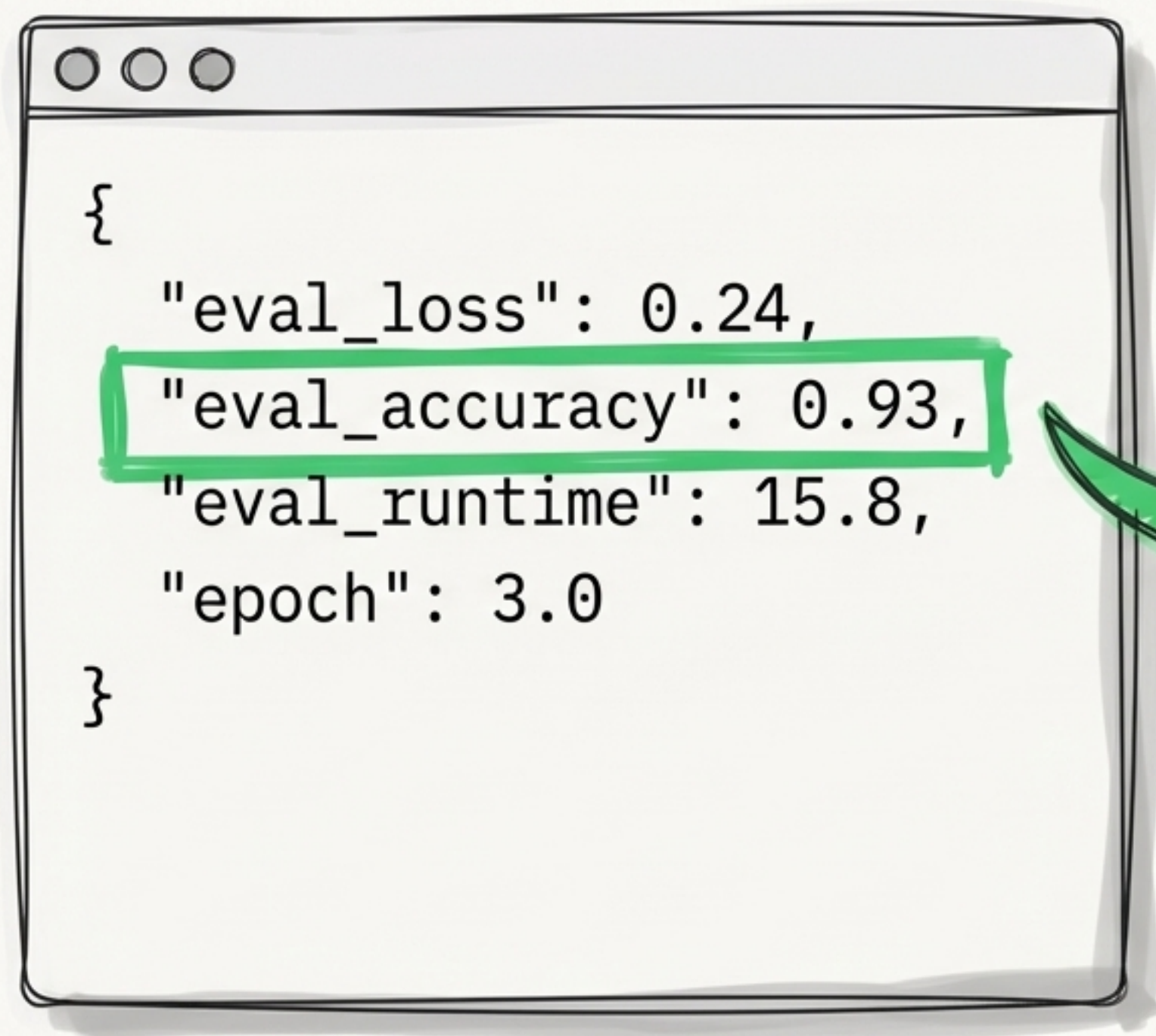


大约需要10分钟

trainer.save_model("./my_finetuned_bert")



训练完成！解读我们的成绩单



93%

准确率 (Accuracy)

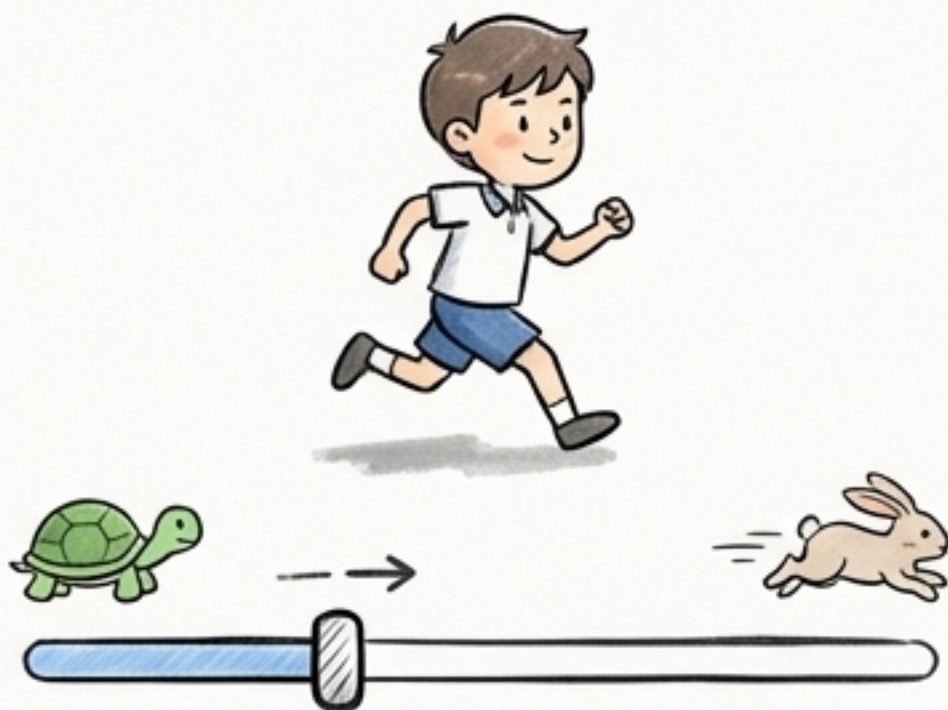
我们的模型在测试集上的预测准确率达到了93%！这意味着100条评论中，有93条的分类是正确的。

损失值 (Loss) : 0.24

这个值越低，代表模型的预测错误程度越小。

深入理解核心超参数 (Hyperparameters)

学习率 (Learning Rate)



类比：像背单词的效率。太快，记得快忘得也快；太慢，效率低。

建议值： $1e-5$ to $5e-5$

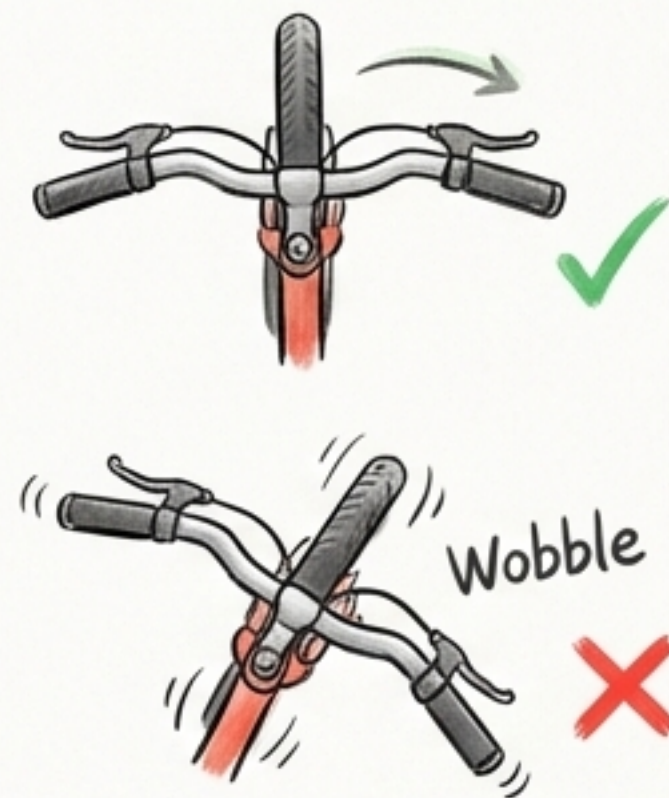
训练轮次 (Epochs)



类比：复习功课的次数。次数太少学不透，次数太多会“死记硬背”（过拟合）。

建议值： 3 to 5

梯度范数 (Gradient Norm)



类比：像骑自行车的转向幅度。幅度太大容易摔倒，太小拐不过弯。

建议值： 5 to 30

效果验证（一）：微调前的BERT

文本	预测结果	置信度
这家酒店的服务员态度真好，下次还来！	积极 (Label: 1) 	0.65
房间干净整洁，设施齐全。	积极 (Label: 1) 	0.68
隔音效果太差了，性价比极低。	积极 (Label: 1) 	0.62

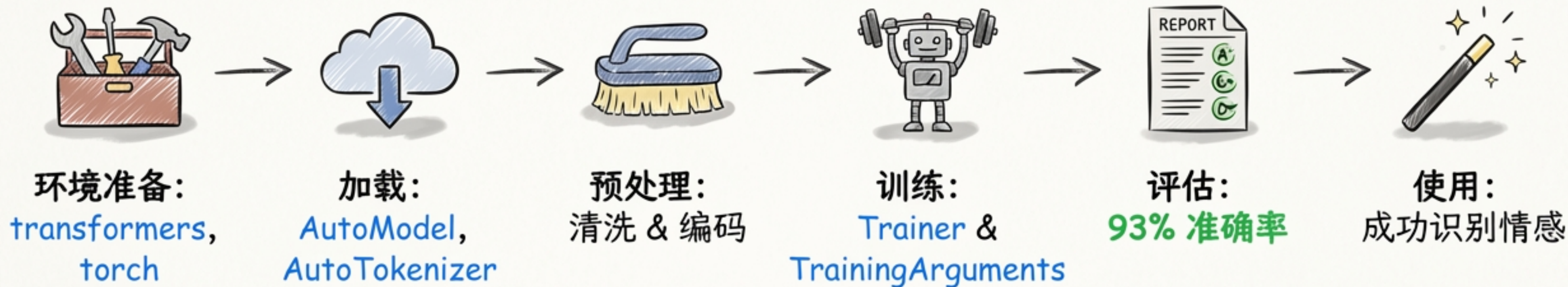
结论：原始BERT模型有基础的语言理解能力，但无法准确识别带有强烈负面情绪的特定领域文本。

效果验证（二）：微调后的模型，脱胎换骨！

文本	预测结果	置信度
这家酒店的服务员态度真好，下次还来！	积极 (Label: 1) 	0.99
房间干净整洁，设施齐全。	积极 (Label: 1) 	0.98
隔音效果太差了，性价比极低。	消极 (Label: 0) 	0.99

结论：微调后的模型不仅成功纠正了错误，并且对所有判断都表现出极高的置信度！

我们的微调之旅回顾



如何更上一层楼？模型优化四法



优化数据质量与数量

增加更多特定领域（如电商、微博）的数据，提高模型的领域适应性。



使用同义词替换

扩充训练数据，让模型理解“超赞”、“棒极了”都等于“好”。



精调超参数

尝试不同的学习率、批次大小和训练轮次，找到最优组合。



升级基础模型

如果基础模型效果不佳，可以尝试更大、更先进的预训练模型。

现在，轮到你了！

你已经掌握了一套可直接复用的项目模板。



将今天学到的流程和代码应用到你自己的场景中，
开始解决真实世界的问题吧！